

```

import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import accuracy_score
import scipy
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(1)
X = np.sort(np.random.randn(100))
y = X - 2 * X**2 + np.random.randn(100)

```

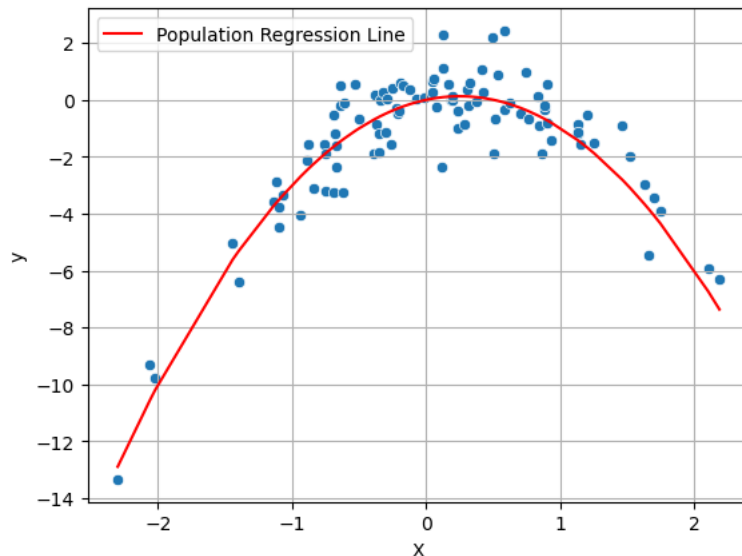
- $n = 100$
- $p = 2$

(b) Create a scatterplot of X against Y . Comment on what you find.

```

ax = sns.scatterplot(x=X, y=y)
ax.plot(X, X-2*X**2, color='r', label='Population Regression Line')
ax.set_xlabel('X')
ax.set_ylabel('y')
ax.grid()
ax.legend()
<matplotlib.legend.Legend at 0x1d09478de80>

```



This scatter plot shows observed data points along with a population regression curve. The regression line is non-linear.

(c) Set a random seed, and then compute the LOOCV errors that result from fitting the following four models using least squares: i. $Y = \beta_0 + \beta_1 X + \epsilon$ ii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$ iii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$ iv. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \epsilon$. Note you may find it helpful to use the `data.frame()` function to create a single data set containing both X and Y .

```

from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

def poly_loocv(data, degree):

    poly = PolynomialFeatures(degree=degree, include_bias=False)
    lm = LinearRegression()

    cv_score = cross_val_score(lm, poly.fit_transform(data['x'][:,
np.newaxis]), data['y'],
                                scoring='neg_mean_squared_error',
cv=len(data['x']))

    return -np.mean(cv_score)
df = pd.DataFrame({ 'y': y, 'x': X })
def poly_loocv(data, degree):
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    lm = LinearRegression()

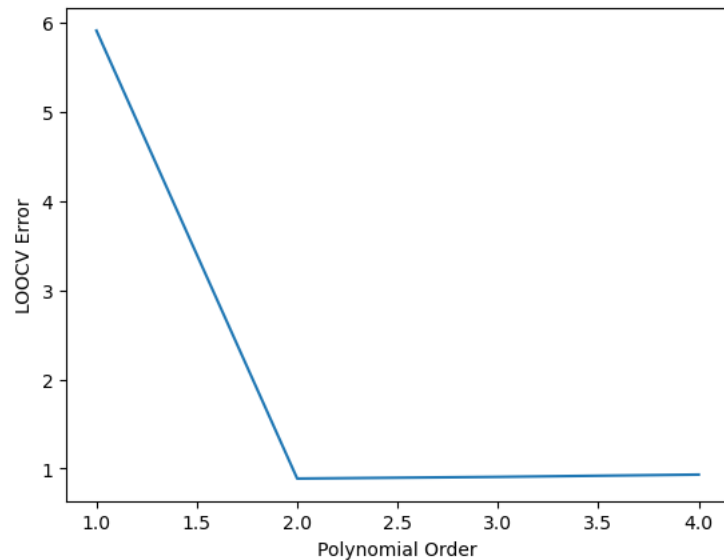
    #  Make X explicitly 2D
    X = data['x'].to_numpy().reshape(-1, 1)    # or: data[['x']].to_numpy()
    y = data['y']                               # y can stay as a Series

    X_poly = poly.fit_transform(X)

    cv_score = cross_val_score(
        lm,
        X_poly,
        y,
        scoring='neg_mean_squared_error',
        cv=len(data['x'])    # LOOCV
    )

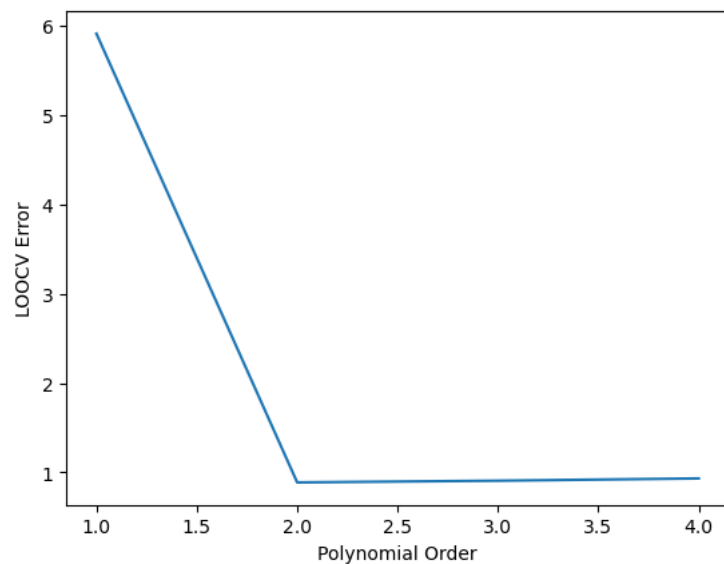
    return -np.mean(cv_score)
np.random.seed(10)
loocv_results = [poly_loocv(df, deg) for deg in range(1, 5)]
loocv_results
[np.float64(5.90663789467059),
 np.float64(0.8889974749237378),
 np.float64(0.907849666352407),
 np.float64(0.9334254546612126)]
plt.plot(range(1,5), loocv_results)
plt.ylabel('LOOCV Error')
plt.xlabel('Polynomial Order')
Text(0.5, 0, 'Polynomial Order')

```



(d) Repeat (c) using another random seed, and report your results. Are your results the same as what you got in (c)? Why?

```
np.random.seed(405)
loocv_results = [poly_loocv(df, deg) for deg in range(1, 5)]
loocv_results
[np.float64(5.90663789467059),
 np.float64(0.8889974749237378),
 np.float64(0.907849666352407),
 np.float64(0.9334254546612126)]
plt.plot(range(1,5), loocv_results)
plt.ylabel('LOOCV Error')
plt.xlabel('Polynomial Order')
Text(0.5, 0, 'Polynomial Order')
```



We get the same results. This curve is not random. Leave one observation out as the test set.

(e) Which of the models **in** (c) had the smallest LOOCV error? Is

this what you expected? Explain your answer.

The LOOCV analysis reveals that the second order polynomial has the smallest error.

(f) Comment on the statistical significance of the coefficient estimates that results from fitting each of the models in (c) using least squares. Do these results agree with the conclusions drawn based on the cross-validation results?

```
df['x_2'] = X**2
df['x_3'] = X**3
df['x_4'] = X**4
df.head()
```

	y	x	x_2	x_3	x_4
0	-13.342828	-2.301539	5.297080	-12.191435	28.059060
1	-9.323992	-2.060141	4.244180	-8.743607	18.013062
2	-9.797305	-2.022201	4.089298	-8.269383	16.722356
3	-5.021465	-1.444114	2.085465	-3.011648	4.349163
4	-6.391812	-1.396496	1.950202	-2.723450	3.803288

```
def stat_significance(df, degree):
    X = sm.add_constant(df.loc[:, ['x', 'x_2', 'x_3', 'x_4']])
    y = df['y']
```

```
    lm = sm.OLS(y, X.loc[:, 'const':f'x_{degree}']).fit()
    print(f'Degree: {degree}')
    print(lm.summary().tables[1])
    print('')
```

```
for i in range(1, 5):
    stat_significance(df, i)
```

Degree: 1

```
=====
=
```

	coef	std err	t	P> t	[0.025	0.975
const	-1.4165	0.234	-6.048	0.000	-1.881	-0.95
x	0.9171	0.264	3.474	0.001	0.393	1.44

```
-----
-
=====
=
```

Degree: 2

=====						
=						
	coef	std err	t	P> t	[0.025	0.97

-						
const	0.0787	0.114	0.689	0.492	-0.148	0.30
x	1.1417	0.106	10.799	0.000	0.932	1.35
x_2	-1.9167	0.084	-22.787	0.000	-2.084	-1.75
=====						

=

Degree: 3

=====						
=						
	coef	std err	t	P> t	[0.025	0.97

-						
const	0.0792	0.115	0.689	0.492	-0.149	0.30
x	1.1549	0.187	6.191	0.000	0.785	1.52
x_2	-1.9174	0.085	-22.587	0.000	-2.086	-1.74
x_3	-0.0056	0.065	-0.086	0.931	-0.134	0.12
=====						

=

Degree: 4

=====						
=						
	coef	std err	t	P> t	[0.025	0.97

-						
const	0.0958	0.140	0.687	0.494	-0.181	0.37
x	1.1555	0.187	6.163	0.000	0.783	1.52
x_2	-1.9685	0.255	-7.733	0.000	-2.474	-1.46
x_3	-0.0037	0.065	-0.057	0.954	-0.134	0.12
x_4	0.0125	0.059	0.213	0.832	-0.104	0.12
=====						

=

